

1. E:

1. Analysis and Comparison The smallest models from previous parts were the Int8-quantized model and the Knowledge Distillation student, both at approximately 4 KB. The best performing were the baseline and KD student at ~98% accuracy. Pruning, despite achieving 70% sparsity, did not reduce file size because TFLite stores zeroed weights as full float32 values.

2. Proposed Strategy Since Int8 quantization and Knowledge Distillation independently reached ~4 KB by attacking the problem from different angles, one compressing weight precision, the other reducing parameter count, combining both should multiply their gains. The proposed strategy is to train a micro architecture using Knowledge Distillation from the teacher, then apply full Int8 post-training quantization on the result.

3. Implementation A micro student with layers Dense(16)→Dense(8)→Dense(3) was trained for 30 epochs using the same distillation loss from part (d), balancing hard and soft teacher labels equally. The trained model was then converted to TFLite with Int8 quantization using a representative dataset for full integer calibration.

4. Evaluation The micro KD + Int8 model achieved approximately 2 KB, half the size of the previous best while maintaining ~96–98% accuracy and strong precision and recall across all three wine classes.

5. Justification Further size reduction was achievable primarily because of the compression: reducing from 3,075 to 387 parameters is an 8× reduction that quantization alone could never achieve. Accuracy was preserved because Knowledge Distillation provides richer soft-label training signals that allow the smaller model to generalize beyond what its parameter count would normally permit. Going further below this point becomes impractical for this dataset with 13 input features, reducing the first layer below 16 neurons creates a bottleneck that begins to hurt accuracy

2. Part (a) — Baseline DNN The baseline model used a Dense(64) → Dense(32) → Dense(3) architecture trained for 20 epochs with batch size 8 and categorical crossentropy loss. Training accuracy reached approximately 70% by the final epoch and test accuracy came out to 90.74%. The float32 TFLite model size was 14.02 KB.

Part (b) — Quantization All three quantized models were evaluated on the same 54-sample test set. Dynamic Range Quantization produced a 8.44 KB model with 91% accuracy, matching the baseline exactly in classification performance. Float16 Quantization came in at 8.77 KB and also achieved 91% accuracy with an identical classification report and confusion matrix to dynamic. Full Integer (Int8) Quantization was the smallest at 7.91 KB but dropped to 85% accuracy, with class_2 recall falling to 0.50 and 7 samples misclassified into class_1. Dynamic and Float16 preserved the baseline

performance perfectly while cutting size roughly in half, whereas Int8 traded some accuracy for the smallest footprint.

Part (c) — Pruning Pruning was implemented manually using a PolynomialDecay schedule from 50% to 70% sparsity over 320 steps since tfmot was incompatible with the TF version in use. The pruned model reached a final sparsity of 69.96% after 20 epochs but validation accuracy was highly unstable, peaking at 72% and dropping as low as 12% across epochs. The resulting TFLite file was 14.05 KB, essentially identical to the baseline at 14.02 KB. Pruning produced no meaningful size reduction because TFLite stores zeroed weights as full float32 values, and the masking disrupted training stability.

Part (d) — Knowledge Distillation The model used a Dense(32) → Dense(16) → Dense(3) architecture with 1,027 parameters, trained for 10 epochs using a custom distillation loss combining hard and soft labels at $\alpha = 0.5$. The teacher's soft labels showed low confidence across classes, which is consistent with a teacher trained without normalization. The student TFLite model was 6.05 KB, a 56.8% reduction from the baseline, but test accuracy collapsed to 63% with class_1 recall at only 0.29.

Part (e) — Further Reduction The smallest model from the previous parts was the KD student at 6.05 KB, though it performed poorly. The Int8 model was the best tradeoff at 7.91 KB and 85% accuracy. Further reduction is possible by combining a micro architecture (Dense(16) → Dense(8) → Dense(3), 387 parameters) with a properly trained teacher and Int8 post-training quantization, projected to reach approximately 3 to 4 KB. Because the teacher was trained without StandardScaler normalization and only achieved 91% accuracy on raw features, the soft labels carried limited inter-class information, making distillation ineffective. Fixing the teacher's training pipeline would have the largest single impact on the student's performance, and combining that improvement with Int8 quantization on the micro student would push size reduction well beyond the current 6.05 KB floor without the accuracy collapse seen in part (d).

3. Question 1 — Learning Blocks on the Free Tier

Edge Impulse provides several pre-defined learning blocks on the free tier, falling into three categories. For supervised learning, the available blocks are Classification (Keras), Regression (Keras), Image Classification via Transfer Learning, Keyword Spotting via Transfer Learning, Object Detection using MobileNetV2 SSD FPN, and Object Detection using FOMO. Example models include a gesture classifier using accelerometer data and a continuous value predictor for sensor regression tasks. For unsupervised learning, Anomaly Detection using K-means, Anomaly Detection using GMM, and Visual Anomaly Detection using FOMO-AD are available, with a typical use case being industrial vibration anomaly detection without labeled failure data. The Classical ML block additionally supports scikit-learn models like decision trees for tabular data classification.

Question 2 — Layer Types in the Classifier Section

The layers available include Dense, Reshape, Flatten, Dropout, Conv1D, Conv2D, MaxPooling1D, MaxPooling2D, and BatchNormalization. Dense applies learned weights across all inputs for general feature combination. Reshape restructures the input tensor into a target shape without altering values. Flatten collapses multi-dimensional outputs into a 1D vector before a final Dense layer. Dropout randomly zeros out neurons during training to reduce overfitting. Conv1D extracts local features from sequential or time-series data. Conv2D does the same over 2D inputs like images or spectrograms. MaxPooling1D and MaxPooling2D reduce spatial dimensions by retaining the maximum value within a sliding window. BatchNormalization normalizes activations across a mini-batch to stabilize training and improve convergence.

Question 3 — Model Compression Options in Deployment

Edge Impulse offers two compression axes at deployment: model version, either quantized int8 or unoptimized float32, and compiler, either TFLite or the EON Compiler, with the EON Compiler also offering a RAM-optimized variant. The EON Compiler compiles neural networks directly to C++ source code, eliminating the need for a generic interpreter at runtime, which reduces RAM usage by up to 55% and flash by up to 35% compared to TensorFlow Lite for Microcontrollers while retaining the same accuracy.

Question 4 — Synthetic Data Generation

Edge Impulse supports synthetic data generation across three input modalities: images, speech, and audio events. For images, DALL-E is used to generate realistic labeled image datasets from text prompts. For speech, Whisper generates human-like voice data for keyword spotting applications. For audio events, ElevenLabs generates environmental sounds such as alarms or glass breaking. Synthetic data is important in machine learning because real-world data collection is often expensive, time-consuming, and difficult to label at scale, particularly for rare events or safety-critical scenarios. Generating synthetic samples allows models to train on diverse and balanced datasets that better reflect edge deployment conditions without requiring physical hardware or extensive field collection.